Delivery Sucre

Práctica de microservicios - COM350

Universidad San Francisco Xavier de Chuquisaca. Gestión 2/2026.

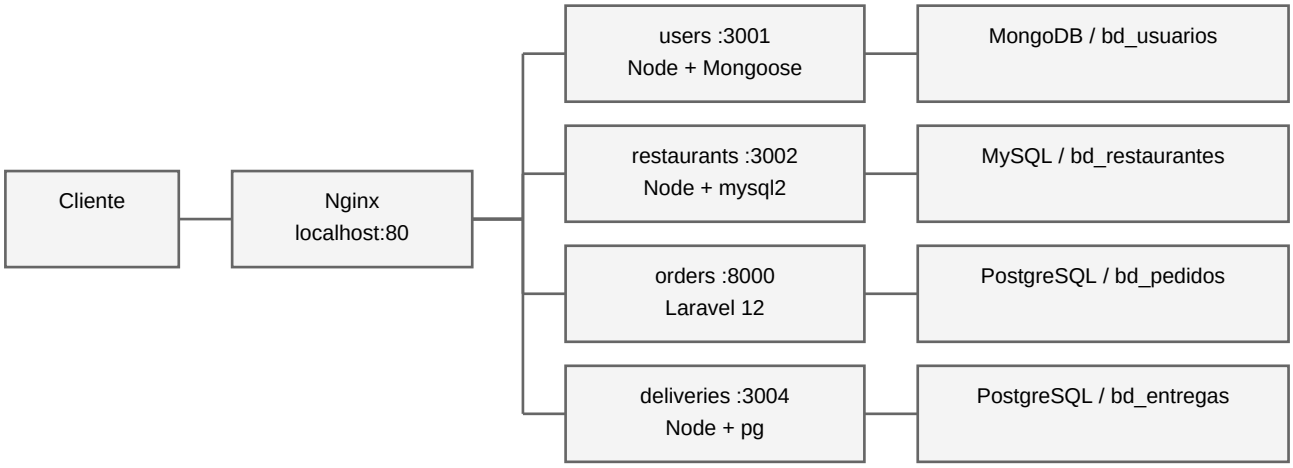
Fecha de elaboración: 28 de octubre de 2026.

Integrantes: completar nombres y apellidos antes de entregar.

1. Qué se implementó

Se completó el proyecto base del docente. Se usó la organización de los ejemplos de 10-27: conexión a la base, modelos con consultas y rutas de Express. Para pedidos se siguió la Parte B con Laravel 12. Para entregas se usó pg y fetch, sin agregar más capas. Las mejoras elegidas fueron CORS y versionado. No se realizó el ejercicio opcional de reseñas.

Arquitectura final



Cada servicio maneja sus propios datos. Orders consulta precios a restaurants y deliveries consulta y actualiza pedidos mediante HTTP. Se reenvía Authorization y los cuatro servicios comparten el secreto JWT. En PostgreSQL se usan dos bases separadas: bd_pedidos y bd_entregas.

Alcance de la comprobación

Se verificó el código localmente: 28 comprobaciones de orders y 26 de deliveries. En estas pruebas se usaron SQLite en memoria y servicios o modelos simulados. También se validó la configuración de Compose, la sintaxis y las rutas Laravel.

Docker Desktop no inició por un error con sailor-ingest.sock. La construcción de imágenes, la integración con las tres bases y nginx, las capturas de Postman, la memoria y el video están pendientes. Este informe no presenta esas comprobaciones como realizadas.

2. Observaciones de la guía

Observación 1: puertos de los microservicios

Los puertos 3001, 3002, 8000 y 3004 están publicados para probar los servicios directamente mientras aprendemos y depuramos. Si se quitan sus bloques ports, nginx todavía puede comunicarse con ellos por appnet y sus puertos internos. El cliente entraría solamente por el puerto 80. Los contenedores no necesitan publicar un puerto al equipo para comunicarse entre sí.

Observación 2: scripts iniciales y volúmenes

Los scripts init se ejecutan cuando el volumen de la base está vacío. Para aplicarlos otra vez desde cero se usa docker compose down -v y luego docker compose up -d --build. Esto borra los datos de los volúmenes de la práctica: usuarios, restaurantes, pedidos y entregas. Un down sin -v conserva los datos. Si hubiera información importante, se haría un respaldo y se modificaría el esquema sin borrar el volumen.

Observación 3: ocultar password en el modelo

User.js elimina password en toJSON. Así no depende de que cada ruta recuerde ocultarlo al devolver un usuario. Incluso un hash no debe entregarse al cliente porque puede facilitar ataques contra contraseñas. La colección agrega una comprobación para que el registro no devuelva password. Esa comprobación en Postman queda pendiente de ejecutar con MongoDB; la regla del modelo ya está en el código base.

Observación 4: referencias entre servicios

user_id y restaurant_id son datos y no claves foráneas, porque pertenecen a bases de otros microservicios. La única FK del pedido es order_items.order_id, dentro de la misma base. Se copian nombres para consultar el pedido sin depender siempre de users o restaurants. Si alguien cambia su nombre o borra un restaurante, la copia puede quedar desactualizada. En el pedido se mantiene la información histórica del momento de compra.

Observación 5: restaurants no disponible

Se captura la excepción de conexión o la respuesta HTTP fallida y POST /orders devuelve 502 en JSON. No se crea un pedido parcial. Esto se comprobó localmente simulando la caída. El ejemplo del PDF no captura esa excepción en store, por lo que puede terminar en 500; aquí se hizo explícito el 502 indicado en el anexo. Se podrían agregar reintentos cortos para los GET y un circuit breaker. Una caché de precios necesitaría una política para evitar vender con valores desactualizados.

3. Casos de entregas C.1

Se probaron las rutas reales de Express y el cliente fetch con un servidor orders simulado. El modelo de deliveries se reemplazó por datos en memoria. Estos resultados comprueban las reglas, pero todavía no prueban las consultas contra PostgreSQL ni el paso por nginx.

Caso	Comprobación	Resultado local
1	Admin asigna pedido en preparación	201, asignado
2	Asigna otra vez el mismo pedido	409
3	Asigna un pedido pendiente	409
4	Asigna un pedido inexistente	404
5	Repartidor lista sus entregas	Solo las propias
6	Otro repartidor cambia estado	403
7	Recogido, en_camino, entregado	200 y orderSync.ok
8	Intenta volver a recogido	409
9	Consulta del pedido al terminar	Estado entregado

También se verificó delivered_at, los permisos de eliminación y la respuesta 502 cuando orders falla. En ese fallo la entrega conserva su estado y se puede reintentar.

Propagación de estados

Recogido y en_camino producen en_camino en orders. Antes del PATCH se consulta el pedido: si ya tiene ese estado, no se vuelve a enviar el mismo cambio. En la prueba del flujo completo se enviaron dos PATCH: uno para en_camino y otro para entregado. Así se evita el 409 que causaría repetir en_camino.

Evidencia disponible

Los resultados completos se guardaron en informe/evidencias/deliveries-local.txt y orders-local.txt. La colección de Postman incluye los nueve casos, otro repartidor y los tokens e identificadores automáticos.

```
26 comprobaciones de deliveries correctas
(modelo en memoria y orders simulado).
28 comprobaciones de orders correctas
(SQLite y HTTP simulado).
```

Pendiente: ejecutar la carpeta 4 del Runner con Docker y agregar capturas reales de asignación, duplicado, permiso denegado y entrega sincronizada. No se incluyen capturas inventadas.

4. Mejoras del gateway C.2

CORS centralizado

Nginx agrega Access-Control-Allow-Origin, Methods y Headers. Se usa always para incluirlos también cuando hay errores. Las solicitudes OPTIONS reciben 204. En este laboratorio se permite cualquier origen y se usa Bearer sin cookies. Se desactivó el middleware CORS de Laravel para mantener una sola configuración.

```
add_header Access-Control-Allow-Origin '*' always;
add_header Access-Control-Allow-Methods
    'GET, POST, PUT, PATCH, DELETE, OPTIONS' always;
add_header Access-Control-Allow-Headers
    'Authorization, Content-Type, Accept' always;
if ($request_method = OPTIONS) { return 204; }
```

La carpeta 5 de Postman envía OPTIONS con Origin y Access-Control-Request-Method. Comprueba HTTP 204 y el header de origen. También consulta orders sin token para comprobar que el 401 conserva CORS. Falta ejecutar estas pruebas con nginx.

Versionado de pedidos

La ruta /api/v2/orders llega al mismo servicio Laravel. Nginx reescribe el prefijo a /api/v1/orders y agrega X-Api-Version: 2. El middleware VerifyJwt registra esa versión en el log. Se mantienen los headers del proxy en esta location porque proxy_set_header tiene reglas de herencia.

```
rewrite ^/api/v2/orders(.*)$ /api/v1/orders$1 break;
proxy_set_header X-Api-Version 2;
proxy_pass http://orders_service;
```

Postman incluye GET /api/v2/orders con token del cliente. Después se revisa docker compose logs --tail=30 orders y se busca Peticion orders con version igual a 2. Es un alias de la API existente, como permite el enunciado. Falta capturar la respuesta y el log en contenedores.

Estado de la verificación

docker compose config --quiet pasó y todas las rutas Laravel se registran. Esto valida la configuración de Compose y la aplicación local, pero no confirma todavía nginx -t, la construcción de las imágenes ni el funcionamiento del gateway. Esas comprobaciones requieren que Docker arranque.

5. Preguntas de análisis C.4

1. Cambio de precio después de crear el pedido

GET /orders/:id muestra el precio y el total guardados al comprar, aunque después cambie el catálogo. unit_price es una copia del precio real que devolvió restaurants en ese momento. Evita cambiar una compra anterior. La prueba local creó un pedido con precio 45, cambió el precio simulado a 80 y comprobó que el pedido seguía mostrando 45 y total 115.

2. Fallo entre deliveries y orders

Si se actualiza primero deliveries y falla el PATCH, las dos bases quedan con estados diferentes. Aquí se consulta y actualiza orders antes de guardar deliveries, para no avanzar la entrega cuando orders está caído. Sin embargo, si orders responde bien y luego falla la escritura local, todavía puede haber una diferencia. No existe una transacción común entre ambos servicios.

Una solución más completa sería Outbox: guardar el estado y un evento en la misma transacción local. Un proceso publica el evento en un broker; orders lo procesa y confirma. Hacen falta reintentos y consumidores idempotentes. Una Saga definiría los pasos y sus compensaciones. No se implementó esa infraestructura porque la práctica pide HTTP sincrónico.

3. JWT en cada servicio o solo en nginx

Validar en cada servicio permite que también se protejan las llamadas internas y el acceso directo a puertos de depuración. A cambio se repite código y se debe mantener el mismo secreto y algoritmo. Validar solo en nginx centraliza la lógica y reduce repeticiones, pero exige cerrar los accesos directos y confiar de forma segura en la identidad que nginx envía. Los permisos de cada operación seguirían siendo responsabilidad del servicio.

4. Dos bases en un mismo PostgreSQL

Sí se mantiene una base por servicio: cada uno usa su base y no lee las tablas del otro. Compartir el servidor ahorra memoria y simplifica el laboratorio. Se pierde aislamiento de recursos y fallos: si PostgreSQL se cae, ambos servicios se afectan. Dos contenedores permitirían mantener, escalar o reiniciar cada motor por separado, con mayor costo de operación.

5. Memoria y tecnología

No se pudo medir porque Docker Desktop no inició. Falta ejecutar docker stats --no-stream después de recorrer el flujo, repetir la medición y anotar users, restaurants, orders y deliveries. No se afirma cuál consumió más sin datos. Cada runtime, framework, cantidad de dependencias y pool de conexiones agrega consumo; por eso se deben medir bajo una carga comparable antes de sacar conclusiones de costos.

6. Dificultades y cierre de la entrega

Dificultades encontradas

- Docker Desktop se cerró al iniciar. El log indicó un fallo al acceder o renombrar sailor-ingest.sock. El motor Linux no quedó disponible. No se restableció Docker ni se eliminaron sus datos.

- En Windows se guardó entrypoint.sh con LF para que funcione con sh dentro del contenedor.

- Las dependencias se resolvieron para PHP 8.3 mediante composer.lock, aunque el PHP local es 8.5. Se agregó mbstring a la imagen además de pdo_pgsql.

- Los correos de Postman se generan por ejecución y los identificadores se guardan automáticamente. Se corrigió order_id en la asignación para enviarlo como número.

- Se evitó repetir el estado en_camino en orders al pasar la entrega de recogido a en_camino.

Qué quedó comprobado

Elemento	Estado
Laravel: migraciones, reglas y permisos	28 comprobaciones locales correctas
Express entregas: nueve casos y fallos	26 comprobaciones locales correctas
Sintaxis PHP y JavaScript	Correcta
Rutas de la API Laravel	5 registradas
Composer y Compose	Configuración validada
Login inicial de admin	Hash bcrypt coincide con admin123
Imágenes, bases y nginx en Docker	Pendiente por fallo de Docker Desktop

Para completar la evidencia de entrega

1. Iniciar Docker y ejecutar docker compose up -d --build. Verificar los ocho contenedores healthy y las migraciones. Probar que down conserva los datos. Hacer el reinicio con -v solo en un entorno de prueba.

2. Ejecutar carpetas 0 a 5 del Runner. Agregar capturas reales de C.1 y C.2 a este informe y mantener un máximo de seis páginas.

3. Medir memoria de los cuatro servicios con docker stats y completar la respuesta C.4.5 con valores observados.

4. Completar integrantes, grabar el video de hasta cinco minutos usando Guion-video.md y subir el proyecto sin node_modules ni vendor al repositorio solicitado.

El proyecto, la colección, las pruebas locales y el guion están preparados. No se grabó un video, no se publicó un repositorio nuevo y no se presentaron resultados de Docker como si se hubieran ejecutado.